LAB: EXTI & SysTick(eval board)

Date: 2026-10-08

Author/Partner: Choi Junghyun

Demo Video: https://youtu.be/wg5xZsDxn28

PDF version:

Introduction

In this lab, you are required to create two simple programs using interrupt:

(1) Display the number counting from 0 to 9 with Button Press

(2) Counting at a rate of 1 second

You must submit

- LAB Report (*.md & *.pdf)
- Zip source files(main*.c, ecRCC2.h, ecGPIO2.h, ecSysTick2.c etc...).
    - Only the source files. Do not submit project files

Requirement

Hardware

- MCU
    - NUCLEO-F411RE
- Eval Board (JKIT)
- Actuator/Sensor/Others:

Software

- PlatformIO, CMSIS, EC_HAL library

Tutorial: Programming Tips

1.Tutorial: Managing library header files

Read how to manage library header files for MCU register configurations. Apply it in your LAB.

{% embed url="https://ykkim.gitbook.io/ec/ec-course/tutorial/tutorial-library-header-files#ec-2024" %}

2.Tutorial: Custom Initialization

Instead of writing initial setting functions for each registers, you can call a user defined function e.g. `MCU_init()` for the commonly used default initialization. Follow the tutorial and apply it in your LAB.

{% embed url="https://ykkim.gitbook.io/ec/ec-course/tutorial/tutorial-custom-initialization" %}

Problem 0: STM-Arduino

{% embed url="https://ykkim.gitbook.io/ec/ec-course/tutorial/tutorial-arduino-stm32/tutorial-arduino-stm32-part1#external-interrupt" %}

We are going to create a simple program that turns LED(LD2) on triggered by **External Interrupt** of user button(BT1) (on NUCLEO-64, not on JKIT)

attachInterrupt()

```
attachInterrupt(digitalPinToInterrupt(pin), ISR, mode)
```

- `digitalPinToInterrupt(pin)`: translate the digital pin to the specific interrupt number.
- `ISR`: a function called whenever the interrupt occurs.
- `mode`: defines when the interrupt should be triggered. (LOW, CHANGE, RISING, FALLING)

Procedure

1. Create a new project under the directory `\repos\EC\lab\`
2. Open *Arduino IDE* and Create a new program named as '**TU_arduino_EXTI.ino**'.
3. Write the following code.
    - Compare this code with the polling method in Lab: GPIO-STM-Arduino

   the arduino code from the GPIO_inout checks pin 3's inside loop, and this code uses interupt(EXTI) allowing the CPU to stay within the loop until a change automatically triggers the blink function.

```
const int btnPin = 3;
const int ledPin =  13;

int btnState = HIGH;

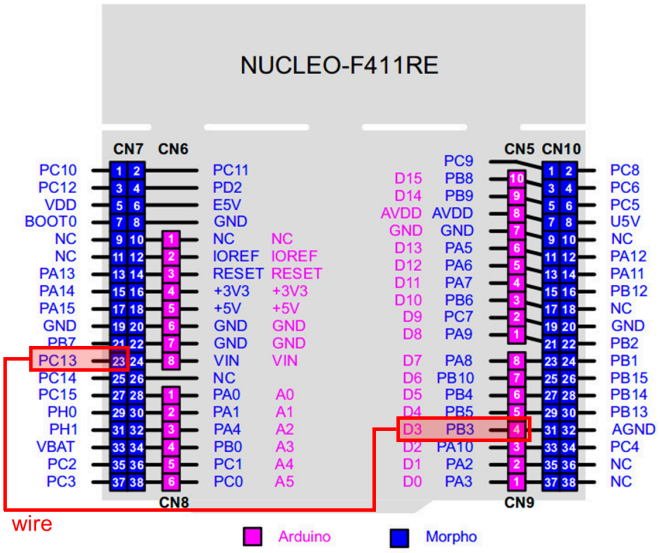
void setup() {
    pinMode(ledPin, OUTPUT);
    pinMode(btnPin, INPUT_PULLUP);
    attachInterrupt(digitalPinToInterrupt(btnPin), blink, CHANGE);
}

void loop() {
    // blank
}


void blink(){
    btnState = digitalRead(btnPin);
```

```
    if (btnState == HIGH)
        digitalWrite(ledPin, LOW);
    else
        digitalWrite(ledPin, HIGH);
}
```

> The user button pin is `PC13`, but this pin cannot be used in arduino. So, you should connect `PC13` to `pinName D3` by using a wire.



4. Click on **upload** button.
5. Whenever the user button(BT1) is pressed (at fall), LED should be ON. When the button is released, the LED should be OFF.

Prelab: EXTI Register Setting

Do the PreLAB: External Interrupt

- It is about creating `ecEXTI.*` library
- Fill-in the blanks in the given header files
- Submit the Prelab report

```
void EXTI_init(PinName_t pinName, uint32_t trig_type, uint32_t priority);
void EXTI_enable(uint32_t pin);  // mask in IMR
void EXTI_disable(uint32_t pin);  // unmask in IMR
uint32_t  is_pending_EXTI(uint32_t pin);
void clear_pending_EXTI(uint32_t pin);
```

Problem 1: Counter on 7-Segment display using EXTI

Problem

Create a code to display the counter-up from 0 to 9 and repeating.

- Count up at the rate of 1 sec.
  - Use `delay_ms()` in the main loop
- Pause the counting when SW2 is pressed
- Resets to zero when the push button is pressed: SW1
  - Must use External Interrupts for SW1, SW2

*** Challenge : Extend the count-up number : 0 to 59**

Procedure

1. Connect the evaluation board (JKIT-NUCLEO) to the MCU.
2. Make sure that your library are in `EC\include\`.
   - **ecRCC2.h, ecRCC2.c**
   - **ecGPIO2.h, ecGPIO2.c**
   - **ecSysTick2.h, ecSysTick2.c**
   - **ecEXTI2.h, ecEXTI2.c**
   - **ecSTM32F4v2.h**
3. Create a new project under the directory `EC\lab\`

- Environment: "**LAB_EXTI".**
- Source File: "**LAB_EXTI.c"**

4. You must modify the `platformio.ini` **,** to add new environment.

Configuration

Configure the MCU GPIO

Function	Port - Pin	Configuration
Button (SW1) on JKIT	PD_2	DIN, Pull-Up
Button (SW2) on JKIT	PA_4	DIN, Pull-Up
7-Segment DOUT	PB_0, PB_1, PB_2, PB_3, PB_4, PB_5, PB_6, PB_7 ('a'~'g''dot', respectively)	Push-Pull, No Pull-up-Pull-down, Medium Speed
Selection of 7-Segment Display	PA_10 (FND_0)	DOUT, Push-Pull,

Circuit Diagram

> You need to include the circuit diagram (if necessary)

Code

Your code goes here.

Explain your source code with the necessary comments.

```
      /*----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : Choi Junghyun
Created          : 2026-09-02
Modified         : 2026-10-08
Language/ver     : C (CMSIS / EC_HAL)

Description      : Problem 1 - 7-Segment Counter using EXTI & SysTick
/----------------------------------------------------------------*/

#include "ecRCC2.h"
#include "ecGPIO2.h"
#include "ecSysTick2.h"
#include "ecEXTI2.h"// needed headers

// pin definitions based on gitbook lab
#define SW1_PIN     PD_2 // define SW1 pin as port D pin 2 (reset button)
#define SW2_PIN     PA_4 // define SW2 pin as port A pin 4 (stop, go button)
#define FND_SEL0    PA_10 // define 7-segment digit enable pin as port A pin 10

// global state variables
volatile uint8_t count = 0;
volatile uint8_t is_paused = 0;

// function prottypes
void setup(void);
void sevensegment_display(uint8_t num);

// setup
void setup(void) {
    RCC_PLL_init(); // system clock to 84MHz
    SysTick_init(); // initialize sysick timer module for millisecond delays

    // initialize 7-Segment data Pins(PB0 to PB7) as output
    for (int i = 0; i < 8; i++) { // loop through 8 data pins
```

```
        GPIO_init(PB_0 + i, OUTPUT); // Configure current pin as digital out
        GPIO_pupd(PB_0 + i, EC_NONE); // set pupdr resistor setting to none
        GPIO_ospeed(PB_0 + i, EC_MEDIUM_SPEED); // set GPIO output speed mode to
medium speed
    } // end of loop

    // initialize 7-Segment digit selector pin(PA10)
    GPIO_init(FND_SEL0, OUTPUT); // PA10 digit enable pin as output
    GPIO_write(FND_SEL0, HIGH);  // PA10 pin HIGH to enable the FND_0 digit
display

    // Initialize SW1(PD2) for reset button EXTI
    GPIO_init(SW1_PIN, INPUT); // configure PD2 pin as digital input
    GPIO_pupd(SW1_PIN, EC_PU); // enable internal pull-up resistor for active low
button SW1
    EXTI_init(SW1_PIN, FALL, 2); // Enable EXTI on PD2, trigger on FALLING edge,
set priority to 2

    // initialize PA4 for pause
    GPIO_init(SW2_PIN, INPUT); // PA4 pin as digital input
    GPIO_pupd(SW2_PIN, EC_PU); // pullup resistor for active low button SW2
    EXTI_init(SW2_PIN, FALL, 1); // enable EXTI on PA4, trigger on FALLING edge,
set priority to 1
}   // End

// Main
int main(void) {  // program starting point
    setup();

    while (1) {  // infinite loop
        // display current number on 7Seg
        sevensegment_display(count); // output the current count digit

        // 1-second interval counting logic
        delay_ms(1000); // pause execution for 1000 milliseconds(1 second) using
SysTick

        if (!is_paused) { // check if the timer is not in a paused state
            count = (count + 1) % 10; // increment count and wrap back to 0 after
reaching 9
        }
    }
}

// 7seg
void sevensegment_display(uint8_t num) {  // function to convert number to 7seg
led
    // 7seg LED bit patterns for digits 0-9 (a, b, c, d, e, f, g, dot)
    const uint8_t number_map[10] = { // array mapping numbers 0-9 to their segment
bit patterns
        0b00111111,// bit pattern to display digit 0
        0b00000110, // bit pattern to display digit 1
        0b01011011,// bit pattern to display digit 2
        0b01001111,// bit pattern to display digit 3
```

```
        0b01100110,// bit pattern to display digit 4
        0b01101101,// bit pattern to display digit 5
        0b01111101, // bit pattern to display digit 6
        0b00000111,// bit pattern to display digit 7
        0b01111111,// bit pattern to display digit 8
        0b01101111 // bit pattern to display digit 9
    };

    if (num > 9) return;    // limit condition, ignores inputs greater than 9

    uint8_t pattern = number_map[num];      // get the bit pattern matching the
requested digit

    // write bits to PB0 through PB7
    for (int i = 0; i < 8; i++) { // loop through each bit from index 0 to 7
        uint8_t bit_val = (pattern >> i) & 0x01; // get the bit value for the
current position
        GPIO_write(PB_0 + i, bit_val); // set the corresponding GPIO pin to output
    }
}

// EXTI2 interrupt handler
void EXTI2_IRQHandler(void) {
    if (is_pending_EXTI(SW1_PIN)) { // check if the interrupt request is from SW1
        count = 0; // reset to 0
        clear_pending_EXTI(SW1_PIN); // clear the interrupt flag
    }
}

// EXTI4 interrupt handler
void EXTI4_IRQHandler(void) {
    if (is_pending_EXTI(SW2_PIN)) { // check if the interrupt request is from SW2
        is_paused = !is_paused; // Toggle flag
        clear_pending_EXTI(SW2_PIN); // clear
    }
}
```

Sample Code

{% code expandable="true" %}

```
/*-----------------------------------------------------------------\
@ Embedded Controller by Young-Keun Kim - Handong Global University
Author           : [ YOUR NAME GOES HERE !!!!!]
Created          : 05-03-2021
Modified         : 09-09-2026
Language/ver     : C++ in VS Code

Description      : Tutorial - [Your Description GOES HERE !!]
/-----------------------------------------------------------------*/
```

```
#include "ecRCC2.h"
#include "ecGPIO2.h"
#include "ecSysTick2.h"
#include "ecEXTI2.h"

#define LED0_PIN   PB_12        //EVAL board JKIT LED0
#define SW2_PIN    PA_4         //EVAL board JKIT SW2


// void led_toggle(PinName_t pinName);  // in ecGPIO2.h

// Initialiization
void setup(void)
{
    RCC_PLL_init();                 // System Clock = 84MHz
    SysTick_init();                 // SysTick Timer Initialization
    // Initialize GPIOB_12 for Output
    GPIO_init(LED0_PIN, OUTPUT);    // LED0 for EVAL board

    // Initialize GPIOA_4 for Input Button
    GPIO_init(SW2_PIN, INPUT);  // INPUT for EVAL board
    GPIO_pupd(SW2_PIN, EC_PU);  // PULL-UP for EVAL board

    // Initialize EXTI PA_4
    EXTI_init(SW2_PIN, FALL, 10);
}


// MAIN  -----------------------------------------
int main(void) {
    setup();
    while (1){
        delay_ms(5000);
    }
}

void EXTI4_IRQHandler(void) {
    if (is_pending_EXTI(SW2_PIN)) {
            led_toggle(LED0_PIN);
            clear_pending_EXTI(SW2_PIN);
    }
}
```

{% endcode %}

Results

Experiment images and results go here

> Show experiment images /results

Add demo video link

Discussion

1. Analyze the result and explain any other necessary discussion.
2. We can use two different methods to detect an external signal: polling and interrupt. What are the advantages and disadvantages of each approach?

> for Polling merits, it offers simplicity and predictable code flow because the hardware continuously checks the input pin state in software, but it wastes massive amounts of CPU cycles and risks missing brief signals if the main execution loop is delayed by other tasks. Interrupt provides simmediate, event driven execution with maximum CPU efficiency, allowing the processor to remain idle or focus on other tasks until a hardware signal state change automatically triggers an ISR. however, interrupts add code complexity, make debugging more difficult, and have mechanical button noises triggering unwanted extra interrupts.

3. What would happen if the EXTI interrupt handler does not clear the interrupt pending flag? Check with your code

> If the EXTI interrupt handler fails to clear the pending flag, the MCU becomes trapped in an infinite interrupt loop and freezes the main program. This happens because the NVIC checks the EXTI Pending Register (EXTI->PR) upon completing the handler. seeing the flag still set to 1, it immediately re-enters the ISR without ever returning control to main()

Reference

Complete list of all references used (github, blog, paper, etc)

Troubleshooting

(Option) You can write a Troubleshooting section